

FRR - PBS Project Report

Pol Garrido Melis¹

¹ Universitat Politècnica de Catalunya - BarcelonaTech

Imanol Munoz-Pandiella

1. Introduction

One of the main concerns in computer graphics is to be able to describe accurately the interactions of light, to be able to represent scenes accurately. However, this is not a trivial problem and involves heavy computations. Then, to be able to solve it in real time, we need to find a balance between physical accuracy and speed.

In this report, I will discuss several solutions, starting from the most trivial ones, like Phong shading, up to more refined solutions based on microfacet theory that aim to approximate how the light interacts at the microscopic level. In particular, we will see a PBR shader for a point light and an IBL one that takes into account the environment to produce more accurate results.

2. Previous Work

One of the main contributions of PBR is that it enables us to model how light behaves with different materials following physical principles. The Cook-Torrance BRDF model [2] provides a foundation for this, combining normal distribution, geometry, and Fresnel components to accurately represent both metallic and dielectric surfaces.

When looking at simpler solutions like Phong shading, we're greatly oversimplifying how light works. In reality, light interacts with materials at a microscopic level. That's what microfacet theory, formalized by Torrance and Sparrow [1], aims to model. This theory views surfaces as collections of tiny mirror-like facets and was later enhanced by Walter et al. [4] with the GGX distribution that produces more realistic specular highlights.

However, the lighting of a scene rarely comes from just point lights in reality. Image-Based Lighting (IBL) techniques, pioneered by Debevec [3], allow us to capture and represent ambient illumination from the environment. The main challenge here is computational complexity, since we would need to integrate over the entire hemisphere of incoming light directions for each surface point.

The split-sum approximation introduced by Karis [6] at Epic Games makes this practical for real-time applications by cleverly separating the rendering equation into components that can be precomputed. These techniques were further improved through importance sampling methods, as described by Krivánek [5], which concentrate samples where they matter most for the final image.

3. Theory

In the previous section we have laid the main foundations of the possibilities that PBR and IBL techniques establish to model the interactions of light by following a physical basis. In this section, we will discuss in detail the mathematical foundation of these techniques to better understand how they

work. Firstly, we need to discuss the reflectance equation, since it is the foundation for all the concepts we will discuss:

$$L_o(p, \omega_o) = \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) n \cdot \omega_i d\omega_i \quad (1)$$

This equation models the output radiance at a given output direction ω_o from a point p in terms of the incoming irradiance from the hemisphere Ω oriented by the normal at point p . This incoming irradiance is scaled by the BRDF: $f_r(p, \omega_i, \omega_o)$ and by the cosine between the normal and incoming ray direction: $n \cdot \omega_i$. As we can see in the equation, the BRDF that we mentioned earlier is modeling the proportion of incoming radiance that gets reflected in terms of the incoming and outgoing light ray direction from point p .

3.1. Cook-Torrance BRDF

As we discussed earlier, the main focus of PBR is to describe a BRDF that holds the principle of energy conservation and that is based on physical properties. In particular, for the Cook-Torrance BRDF, they split the light behavior into two parts, the diffuse and specular terms:

$$f_r = k_d f_{\text{lambert}} + k_s f_{\text{cook-torrance}} \quad (2)$$

Then, if we expand the expressions, we can see that the distribution functions that we mentioned in the previous section appear. The *DFG* terms are the ones responsible for modeling the materials based on the microfacet theory given a roughness term.

$$f_r = k_d \frac{c}{\pi} + \frac{DFG}{4(\omega_o \cdot n)(\omega_i \cdot n)} \quad (3)$$

The first part of the BRDF is the diffuse term, the one responsible for refracted light. In this expression, it is modeled in a Lambertian fashion, where c is the albedo and π a normalization term. There are more elaborate expressions for the diffuse term, but none of them are as cheap as this one, and we get marginal improvements with more complex expressions.

Now, the more interesting part of the BRDF comes in the specular term, the one responsible for reflected light. There are multiple ways to define each of the *DFG* terms, so we'll go through the ones implemented in this project.

3.1.1. D term. The normal distribution function is modeling how the microfacets align with the half-view vector. This proportion of aligned microfacets depends on the roughness of the surface. Specifically, if we are modeling a smooth surface (with low roughness), the area of aligned microfacets will be small, returning a sharp and concentrated reflection. On the other hand, for a really rough surface, it is more likely to find microfacets mostly aligned with the halfview vector but much more dispersed over the surface.

$$D_{GGX}(n, h, \alpha) = \frac{\alpha^2}{\pi((n \cdot h)^2(\alpha^2 - 1) + 1)^2} \quad (4)$$

3.1.2. F term. The Fresnel term returns the fraction of light that gets reflected depending on the viewing angle against the surface. Any surface has a base value of reflectivity when viewed up front, but this reflectivity increases as we look more and more from grazing angles, arriving to fully reflect light at 90 degrees. This progression follows the Fresnel equation, but since this equation would be too complex for real-time applications, it can be approximated by the following expression, the Fresnel-Schlick approximation:

$$F(v, h) = F_0 + (1 - F_0)(1 - (v \cdot h))^5 \quad (5)$$

3.1.3. G term. Finally, the geometry distribution function term accounts for the self-shadowing between the microfacets, based on the roughness of the surface, similarly to how the D term also depended on it. One of the widely used approximations is the Schlick-GGX formula:

$$G_{SchlickGGX}(n, v, k) = \frac{n \cdot v}{(n \cdot v)(1 - k) + k} \quad (6)$$

Since we need to account for geometry obstruction and geometry shadowing, the final expression of the G term is the following, where we account for each case respectively:

$$G(n, v, l, k) = G_1(n, v, k)G_1(n, l, k) \quad (7)$$

3.2. Final reflectance equation

Finally, if we substitute the Cook-Torrance BRDF into the reflectance equation, we get the expression that defines a PBR approach to compute the light interactions. That will be the expression used in the implementation of this project.

$$L_o(p, \omega_o) = \int_{\Omega} \left[k_d \frac{c}{\pi} + \frac{DFG}{4(\omega_i \cdot n)(\omega_o \cdot n)} \right] L_i(p, \omega_i) n \cdot \omega_i d\omega_i \quad (8)$$

Remember that the Fresnel term is giving us the k_s term; that is why it does not appear explicitly in the formula.

3.3. Image-based lighting

With the previous expression, we can represent direct light sources as point lights efficiently, basing their interactions on physical properties. In a scene with a small number of direct point lights, the integral would effectively be a finite sum of light contributions. However, if we wanted to represent the ambient contribution of the environment with its diffuse and specular term, the expression could not be solved in real time since we would need to take into account the full hemisphere Ω of incoming irradiance for each point of the surface instead of a small number of incoming directions.

IBL is a way of solving this problem. The first important observation is to see that we can split the original reflectance equation that we obtained in the previous section into the diffuse and specular parts, since they are independent.

$$L_o(p, \omega_o) = \underbrace{\int_{\Omega} k_d \frac{c}{\pi} L_i(p, \omega_i) n \cdot \omega_i d\omega_i}_{\text{Diffuse Component}} + \underbrace{\int_{\Omega} \frac{DFG}{4(\omega_i \cdot n)(\omega_o \cdot n)} L_i(p, \omega_i) n \cdot \omega_i d\omega_i}_{\text{Specular Component}} \quad (9)$$

First, we will focus on the diffuse part of the function. Since the Lambert term is constant, we can extract it from the integral, and we get this function:

$$L_{\text{diff}}(p, \omega_o) = k_d \frac{c}{\pi} \int_{\Omega} L_i(p, \omega_i) n \cdot \omega_i d\omega_i \quad (10)$$

Now, we have an expression that just depends on the incoming light direction since we are making the simplification that the environment is distant relative to the surface point p . Another crucial aspect of IBL is that we are encoding our environment as a cubemap, so now from this expression we could precompute the diffuse irradiance map by convoluting this cubemap for each outgoing direction ω_o (that would be each texel of the cubemap) in terms of all the ω_i of its hemisphere. However, even though this is a precomputation, sampling the whole hemisphere for each texel of the environment is still computationally expensive, so there are two alternatives: use a structured sampling method by dividing the hemisphere into multiple solid angles, or use a Monte Carlo method to obtain an approximation of the whole hemisphere result.

The second part of the original expression accounts for the specular component, but now it is not as easy as for the diffuse part. The difference resides in that the BRDF component can not be extracted from the integral as we did for the Lambert term because it depends on both the light and view directions. However, if we make some sacrifices, we can indeed pre-compute it as we did for the diffuse part using split sum approximation.

3.3.1. Split sum approximation. What we are achieving is splitting the specular part of the computation into two parts that can be precomputed independently, and then we will use them together in a shader.

$$L_{\text{spec}}(p, \omega_o) = \underbrace{\int_{\Omega} L_i(p, \omega_i) n \cdot \omega_i d\omega_i}_{\text{Pre-filtered Environment Map}} \cdot \underbrace{\int_{\Omega} f_r(\omega_i, \omega_o) n \cdot \omega_i d\omega_i}_{\text{BRDF Integration Map}} \quad (11)$$

This expression, which has been simplified to fit on the page, has a first part that is equivalent to the one of the diffuse irradiance map, but the difference resides in the sampling of the hemisphere, which for this expression will be biased as a function of the surface roughness. Specifically, to solve the pre-filtered environment map, we will be using importance sampling that I will detail in the implementation section.

Finally, for the BRDF integration map, we need to account for the F and G terms of the original BRDF, since the D is already encoded on the left part of the expression that we have precomputed. For this project, I won't be computing the proper lookup texture that would encode the BRDF response given an input roughness value and the angle between the normal and the light direction. Instead, to simplify the second part of the specular contribution, we will only compute it for the view direction, since it is where the specular effects are more noticeable. This way we only need to precompute the pre-filtered environment map and the diffuse irradiance map for our implementation of IBL.

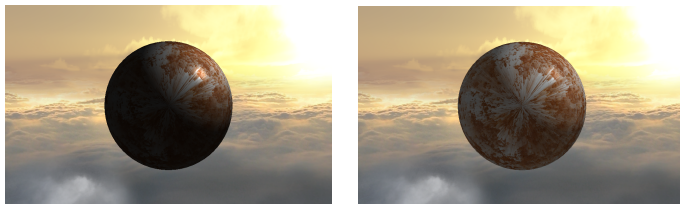
4. Implementation

Due to the short length of the report, I will focus on the shader code, and we will go over the basis of the project in a more broad way. The first functionalities of the project are being able to render an object with a texture mapped into it using Phong shading. Since this is a simple enough shader, I will just focus on the topic of gamma correction that already appears:

```
/* in,outs and uniforms */
void main (void) {
    /* previous computations */
    vec3 tex = pow(texture(current_texture, TexCoord).rgb,vec3(2.2));
    vec3 FinalColor = (ambient + diffuse + specular) * tex;
    vec3 gammaResult = pow(FinalColor, vec3(1.0/2.2));
    frag_color = vec4(gammaResult, 1.0);
}
```

Code 1. Phong shader with texture mapping

As a quick reminder, in Phong shading we multiply the sum of a constant ambient term with the diffuse component that is the light color weighted by the cosine between the light direction and the normal of the fragment and a specular term that is instead weighted by the cosine between the reflected light vector and the view direction. What is relevant in this code is that the albedo extracted from the texture is elevated by some exponent. This operation is accounted for by the gamma correction, since most textures are in gamma space, where the increase of light intensity is not linear. For light computations this is not right, so we have to account for that and later go back to gamma space for the output. Since the human eye is more sensitive to changes in darker intensities of light, the monitors are manufactured to work in this space, making the result more pleasing.



(a) No gamma correction

(b) With gamma correction

Figure 1. Comparison of rendering with and without gamma correction

Since we will be using cubemaps for IBL, there is also an option to render a cubemap of the environment and make the object behave like a perfect mirror by reflecting the observer direction by the normal of the surface and sampling the corresponding texel of the cubemap in world space.

```
/* in,outs and uniforms */
void main (void) {
    vec3 obsDir = normalize(FragPos);
    vec3 reflObsDir = reflect(obsDir,nm_Normal);
    vec3 reflRayWorldSpace = vec3(inv_view * vec4(reflObsDir,1.0));
    frag_color = vec4(textureLod(sky,reflRayWorldSpace,0).rgb,1.0);
}
```

Code 2. Reflection shader

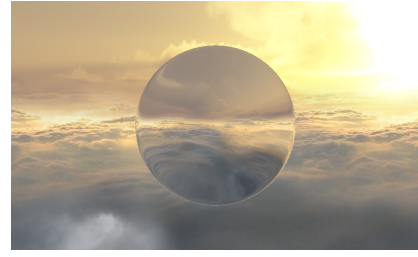


Figure 2. Reflection shader render

Finally, let's go over the PBR and IBL shaders. Firstly, we can see the implementation of D, F, G terms explained in the theory section. For the most part the implementation is a translation from the mathematical formulas, but there are some details to cover. For example, the roughness value that is used in the computation of the normal distribution function is clamped to the range $[0.01, 0.99]$. This is mostly *artistic*, but it makes the slider for the roughness selection more intuitive, since for a roughness of 0 the specular reflection is so small that it can flicker in and out, and for a roughness of one the surface becomes textureless. The other important point is the addition of another formulation for the Fresnel function. This one is used for the IBL computation, since there is no half-view vector defined for the multiple light directions. Since there is no halfview vector and we must use the normal and the view vector, for the standard function, the surface would appear to be reflective, so the roughness term helps to mitigate that by taking into account the nature of the surface.

```
float D(vec3 normal, vec3 h, float r) {
    float r_clamped = min(max(r, 0.01), 0.99);
    float s_roughness = r_clamped * r_clamped;
    float n_times_h = max(dot(normal, h), 0.0);
    float s_n_times_h = n_times_h * n_times_h;
    float denominator = s_n_times_h * (s_roughness - 1) + 1;
    float f_denominator = PI * (denominator * denominator);
    float numerator = s_roughness;
    return numerator / f_denominator;
}

float G(vec3 normal, vec3 dir, float k) {
    float n_times_dir = max(dot(normal, dir), 0.0);
    float denominator = n_times_dir * (1 - k) + k;
    return n_times_dir / denominator;
}

vec3 F(vec3 v, vec3 h, vec3 F0) {
    float cosTheta = max(dot(v, h), 0.0);
    return F0 + (1.0 - F0) * pow(1.0 - cosTheta, 5.0);
}

// Schlick's approximation for IBL
vec3 FresnelSchlickRoughness(float cosTheta, vec3 F0, float roughness) {
    return F0 + (max(vec3(1.0 - roughness), F0) - F0) * pow(1.0 - cosTheta, 5.0);
}
```

Code 3. Cook-Torrance terms

For the point light computation of the shader, we just need to apply the formula explained in the theory section. It's important to note that the base reflectance term (F_0) is obtained from the term passed as a uniform tint towards the albedo of the surface, given its metalness. This way we can use the Fresnel equation for metallic surfaces. Also, since metallic surfaces don't diffuse light, the kD term can be nullified.

```
vec3 F_direct = F(v, halfway, F0);
vec3 kS_direct = F_direct;
vec3 kD_direct = vec3(1.0) - kS_direct;
kD_direct *= 1.0 - u_metalness;
float s_roughness_p1 = (u_roughness + 1.0) * (u_roughness + 1.0);
float k_direct = s_roughness_p1 / 8.0;
vec3 numerator = D(nm_Normal, halfway, u_roughness) *
    G(nm_Normal, l, k_direct) *
    G(nm_Normal, v, k_direct) * F_direct;
```

```
float denominator = 4.0 * max(dot(nm_Normal, v), 0.0) *
    max(dot(nm_Normal, l), 0.0);
vec3 specular = numerator / max(denominator, 0.001);
float NdotL = max(dot(nm_Normal, l), 0.0);
Lo += (kD_direct * diffuse_part(u_Color) + specular) * radiance *
    NdotL;
```

Code 4. PBS shader

Now, for the computation of the IBL ambient term with its diffuse and specular contribution, we just sample the diffuse contribution by reflecting the view vector in world space and multiplying it by the Lambert term, and for the specular part, since we are not using a BRDF integration map, I just compute the geometric term in the direction of the view as discussed in the theory section.

```
float k_ibl = (u_roughness * u_roughness) / 2.0;
float G_IBL = G(worldNormal, V, k_ibl);
vec3 diffuse_ibl = irradiance * diffuse_part(u_Color);
vec3 specular_ibl = specularIrradiance * (F_ambient * G_IBL) /
    (4.0 * NdotV);
vec3 ambient = kD_ambient * diffuse_ibl + specular_ibl;
```

Code 5. IBL shader

Finally, the precomputation of the diffuse irradiance map and the pre-filtered cubemap for the specular part of the IBL computation have been coded in C++ and OpenMP to speed up the computation. As discussed in the theory section, you could follow a structured approach or use Monte Carlo sampling. Since for the specular part of the precomputation we have to perform importance sampling in terms of the roughness, I chose to also apply Monte Carlo sampling for the computation of the diffuse irradiance map using a cosine-weighted PDF.

The pre-filtered environment map implementation leverages several key sampling techniques. Firstly, the Hammersley Sequence generates well-distributed 2D points based on the Van der Corput sequence, and secondly, GGX Importance Sampling biases the sampling directions toward where the GGX normal distribution function is highest.

```
inline glm::vec3 importanceSampleGGX(float u, float v, const glm::
    vec3& normal, float roughness){
    float s_roughness = roughness * roughness;
    float phi = 2.0f * glm::pi<float>() * u;
    // Roughness-weighted distribution
    float cosTheta = sqrt((1.0 - v) / (1.0 + (s_roughness *
        s_roughness - 1.0) * v));
    float sinTheta = sqrt(1.0 - cosTheta*cosTheta);
    float x = sinTheta * cos(phi);
    float y = sinTheta * sin(phi);
    float z = cosTheta;
    glm::vec3 up = std::abs(normal.z) < 0.999f ? glm::vec3(0.0f, 0.0f
        , 1.0f) : glm::vec3(1.0f, 0.0f, 0.0f);
    glm::vec3 tangent = glm::normalize(glm::cross(up, normal));
    glm::vec3 bitangent = glm::normalize(glm::cross(normal, tangent));
    glm::mat3 TBN(tangent, bitangent, normal);
    return TBN * glm::vec3(x, y, z);
}
```

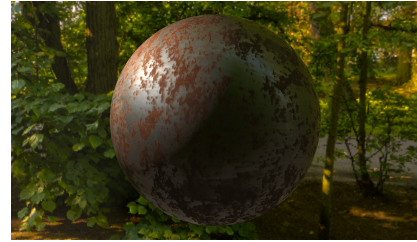
Code 6. Importance sampling

In principle, we are generating a random vector direction biased towards the z-axis in tangent space as a function of roughness and then transforming to world space coordinates to be able to sample the environment. In the results section we can see the results for a given face of the pre-filtered cubemap.

5. Results

In this section, we can see a comparison for a given rusty metal ball in a garden environment going from a simple texture map with Phong shading up to the IBL implementation. Also, to ease the comparison, for the IBL render, the point light present

for the PBR shader is also present, so both the environment and the point light are taken into account.

**(a) Texture mapping with Phong****(b) PBS with a point light****(c) IBL with a point light****Figure 3. Comparison of rendering techniques**

We can see that the major step up in quality comes from having a physically based shading, but taking into account the environment also gives a significant improvement in quality from a constant ambient term.

**(a) Diffuse irradiance****(b) Pre-filtered (mip 1)****(c) Environment****Figure 4. Results for the precomputation**

Finally, we can see some results for the diffuse irradiance maps using a cosine-weighted distribution and the pre-filtered environment maps that weight the distribution in terms of the roughness value. In particular, here the first MIP level is shown.

6. Conclusions

In this project I have explored how a physically based illumination model could be implemented based on the microfacet theory and how to extend the computations for a simple point light to finally take the environment into account. Seeing the progression from basic Phong shading, it's really impressive to see how much the results can improve without compromising the performance. However, there are some areas that could be extended; for instance, ambient occlusion has not been taken into account, which would improve the results, especially when tackling more complex models. Another improvement would be to add support for HDR environment maps to significantly improve the results for IBL, especially in scenes with high contrast. Finally, for this project the BRDF integration map has been approximated by a simple geometry term, but this could also be precomputed.

References

- [1] K. E. Torrance and E. M. Sparrow, “Theory for off-specular reflection from roughened surfaces”, *Journal of the Optical Society of America*, vol. 57, no. 9, pp. 1105–1114, 1967. DOI: [10.1364/JOSA.57.001105](https://doi.org/10.1364/JOSA.57.001105).
- [2] R. L. Cook and K. E. Torrance, “A reflectance model for computer graphics”, *ACM Transactions on Graphics*, vol. 1, no. 1, pp. 7–24, 1982. DOI: [10.1145/357290.357293](https://doi.org/10.1145/357290.357293).
- [3] P. Debevec, “Image-based lighting”, *IEEE Computer Graphics and Applications*, vol. 22, no. 2, pp. 26–34, 2002. DOI: [10.1109/38.988744](https://doi.org/10.1109/38.988744).
- [4] B. Walter, S. R. Marschner, H. Li, and K. E. Torrance, “Microfacet models for refraction through rough surfaces”, in *Proceedings of the 18th Eurographics Conference on Rendering Techniques*, ser. EGSR’07, Eurographics Association, 2007, pp. 195–206.
- [5] J. Krivánek and M. Colbert, “Real-time shading with filtered importance sampling”, *Computer Graphics Forum*, vol. 27, no. 4, pp. 1147–1154, 2008. DOI: [10.1111/j.1467-8659.2008.01259.x](https://doi.org/10.1111/j.1467-8659.2008.01259.x).
- [6] B. Karis, “Real shading in unreal engine 4”, in *SIGGRAPH 2013 Course: Physically Based Shading in Theory and Practice*, ACM SIGGRAPH, 2013.